

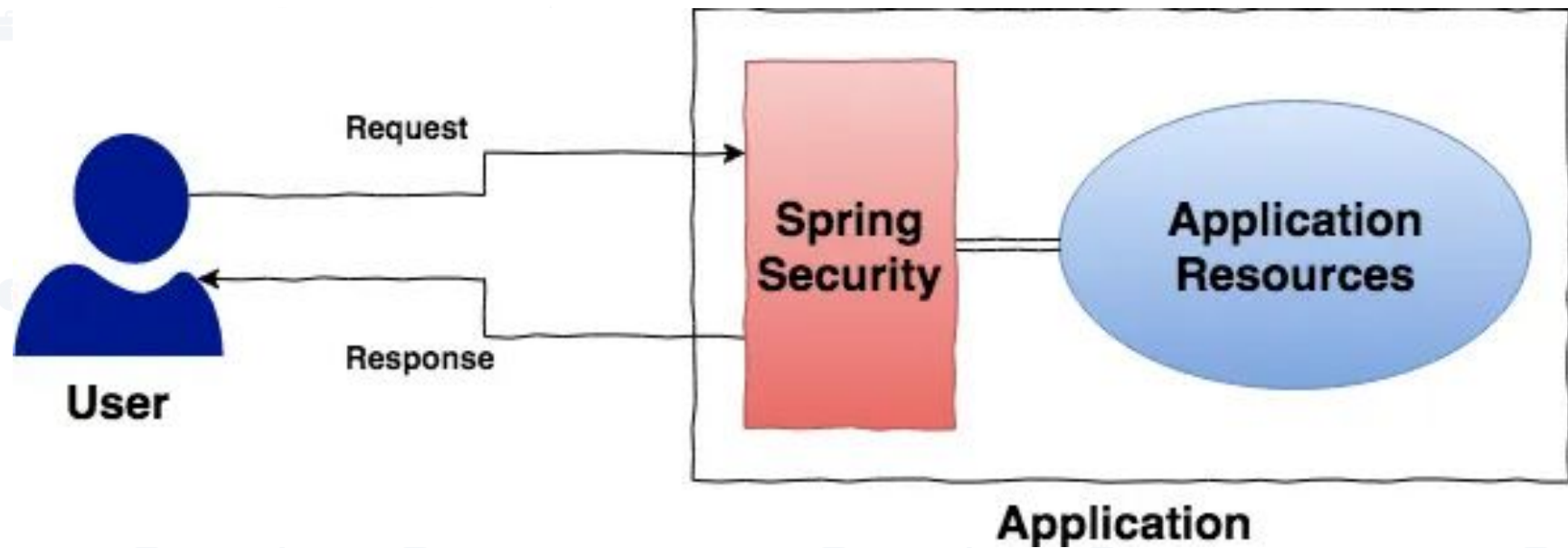


Lesson 40

20.11.2025



spring
SECURITY





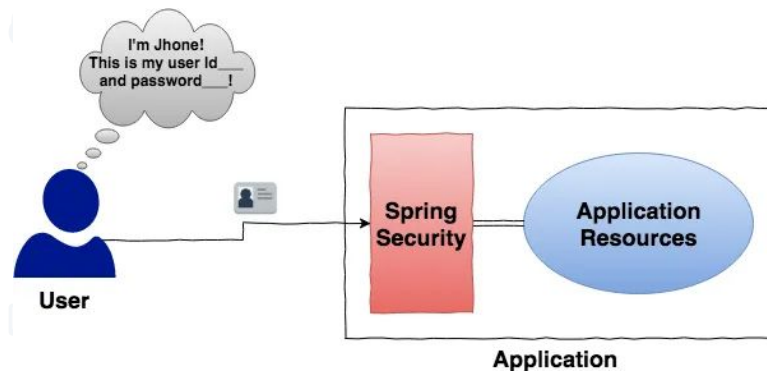
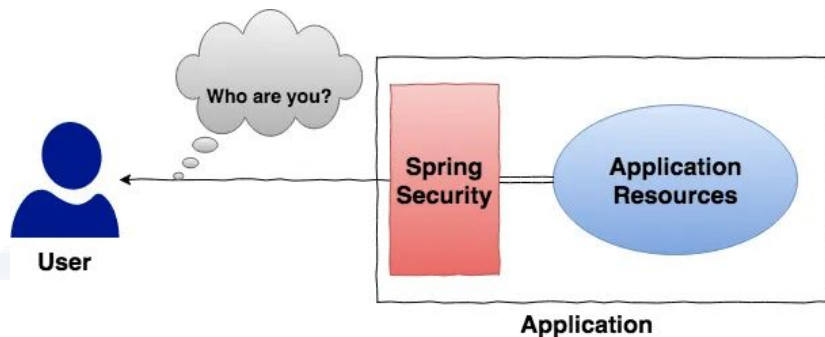
За допомогою Spring Security можна реалізувати різні функції безпеки, такі як:

- Функціонал входу та виходу з системи.
- Аутентифікація та авторизація.
- Дозвіл або блокування доступу до URL для користувачів.
- Реалізація Single Sign-On, LDAP тощо.
- Впровадження OAuth.
- Надання доступу на основі ролей.

1. Аутентифікація

Аутентифікація — це запит до користувача: «Хто ти?». Тобто аутентифікація — це процес визначення та ідентифікації користувача, який хоче отримати доступ до застосунку.

У Spring Security система ставить це запитання користувачу, щоб перевірити його особу, і користувач має надати свої дійсні дані. Під такими даними зазвичай мається на увазі ім'я користувача та пароль, які відомі лише цьому користувачу.



2. Авторизація

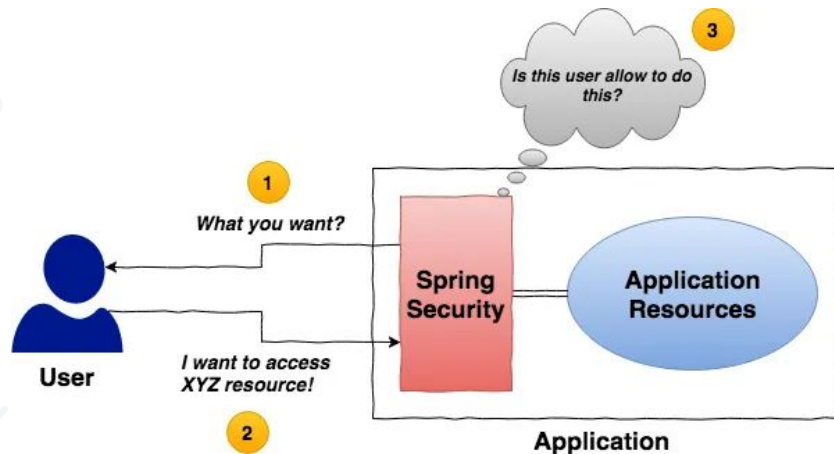
Авторизація — це процес надання користувачу дозволу виконувати певні дії в застосунку.

Простими словами: Spring Security запитує користувача — *«Що ти хочеш зробити?»*. Користувач відповідає: *«Я хочу виконати дію XYZ!»*.

Тоді Spring Security перевіряє: *«Чи має цей користувач право це робити?»*.

Якщо так — дозволяє виконати дію, якщо ні — забороняє.

Отже, під час авторизації ми перевіряємо, чи має вже автентифікований користувач право виконувати певну дію в застосунку.



3. Authenticated Principal (автентифікований суб'єкт)

Principal — це особа, яка була ідентифікована під час процесу аутентифікації. Тобто це *поточний авторизований користувач*.

У Spring Security principal містить ім'я користувача, який наразі увійшов у систему. Він представлений інтерфейсом **AuthenticatedPrincipal**, який має лише один метод — `getName()`, що повертає ім'я автентифікованого користувача:

Після успішної аутентифікації застосунок «прикріплює» principal до користувача і зберігає його, щоб пам'ятати, хто виконує запити.

Саме тому користувачу не потрібно входити в систему заново для кожного HTTP-запиту.

4. Granted Authority (наданий дозвіл)

Під час авторизації ми бачимо, що різні користувачі можуть мати різні рівні доступу. Наприклад, користувач А має доступ лише до функцій F1 і F2, а користувач В — тільки до F1.

Як це контролювати? Очевидно, власник застосунку повинен задати ці відповідності ще до того, як користувачі увійдуть у систему.

У Spring Security такі дозволи для конкретних користувачів називаються **Granted Authorities** — *надані повноваження*.

Тобто для доступу до певної функції застосунку користувач повинен мати відповідний дозвіл.

Наприклад, у застосунку є три можливі дії: перегляд даних, оновлення даних та їх видалення.

Тоді можна створити три авторитети:

- **view_data**
- **update_data**
- **delete_data**

І призначити їх користувачам:

- Користувач А має дозволи: *view_data + update_data*
- Користувач В має дозволи: *view_data + update_data + delete_data*



5. Ролі

Роль — це група дозволів (authorities).

У попередньому прикладі є проблема: складно вручну призначати кожному користувачу набір окремих дозволів. Тому виникає логічне рішення: згрупувати потрібні дозволи та дати цій групі певне ім'я. Таке ім'я і називається **роллю**.

Приклад:

- **ADMIN** роль: *view_data + update_data + delete_data*
- **USER** роль: *view_data + update_data*

Тепер замість призначення кожного дозволу окремо, ми можемо просто призначити користувачу роль.

- Користувач А має роль **USER**
- Користувач В має роль **ADMIN**

6. Фільтри

Фільтр — це об'єкт, який виконується під час попередньої та післяобробки HTTP-запиту.

Фільтри — базова концепція у сервлет-додатках. Вони можуть виконувати різні задачі: логування, конвертацію даних, компресію, шифрування та дешифрування, валідацію вхідних даних тощо — і роблять це для *кожного* запиту до сервера.

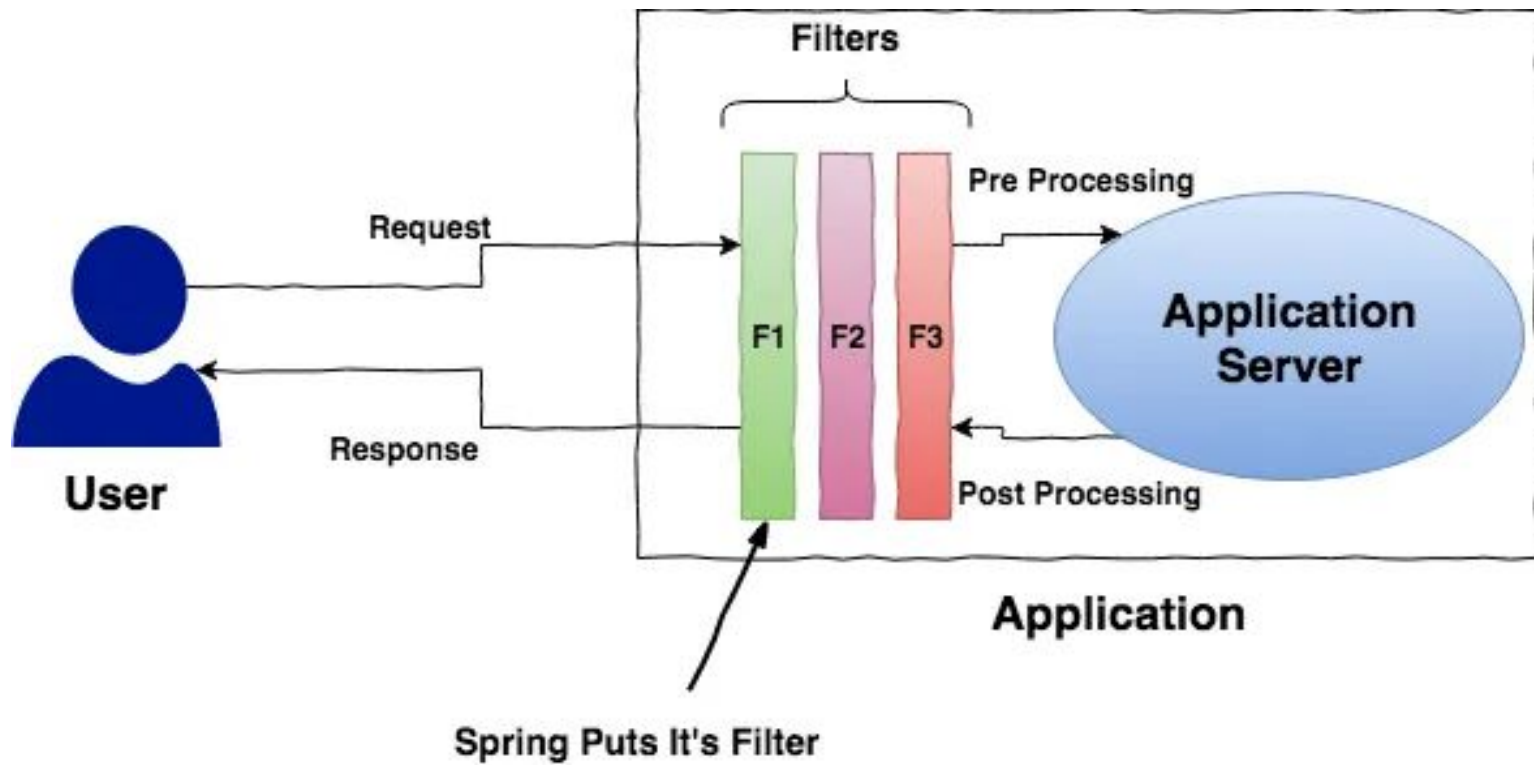
Фільтри знаходяться між клієнтом і застосунком, перехоплюючи кожен запит і виконуючи потрібні дії.

У Spring Security працює та сама логіка: він створює ланцюжок сервлет-фільтрів між користувачем і застосунком та аналізує кожен запит відповідно до налаштованих правил — дозволити чи заборонити.

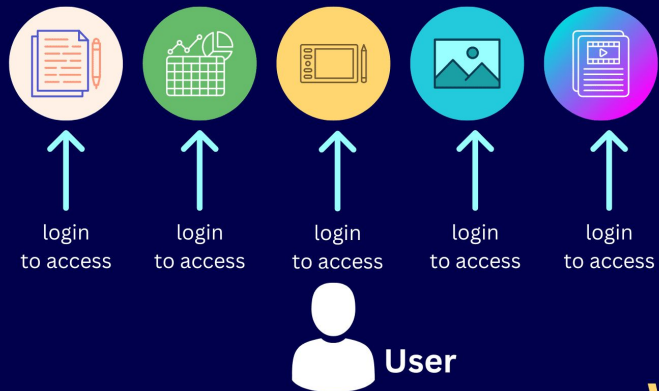
Один із прикладів фільтрів Spring Security — **UsernamePasswordAuthenticationFilter**, який отримує облікові дані з запиту та створює об'єкт **Authentication** із цими даними.

Що роблять фільтри Spring Security за замовчуванням, без жодних налаштувань?

- Додають вимогу обов'язкової аутентифікації для всіх URL.
- Не вимагають аутентифікації для сторінки помилки.
- Автоматично додають стандартну форму логіну.
- Обробляють помилки входу.
- Генерують стандартні ім'я користувача та пароль:
 - логін: **user**
 - пароль: генерується автоматично щоразу при запуску та виводиться в логах.

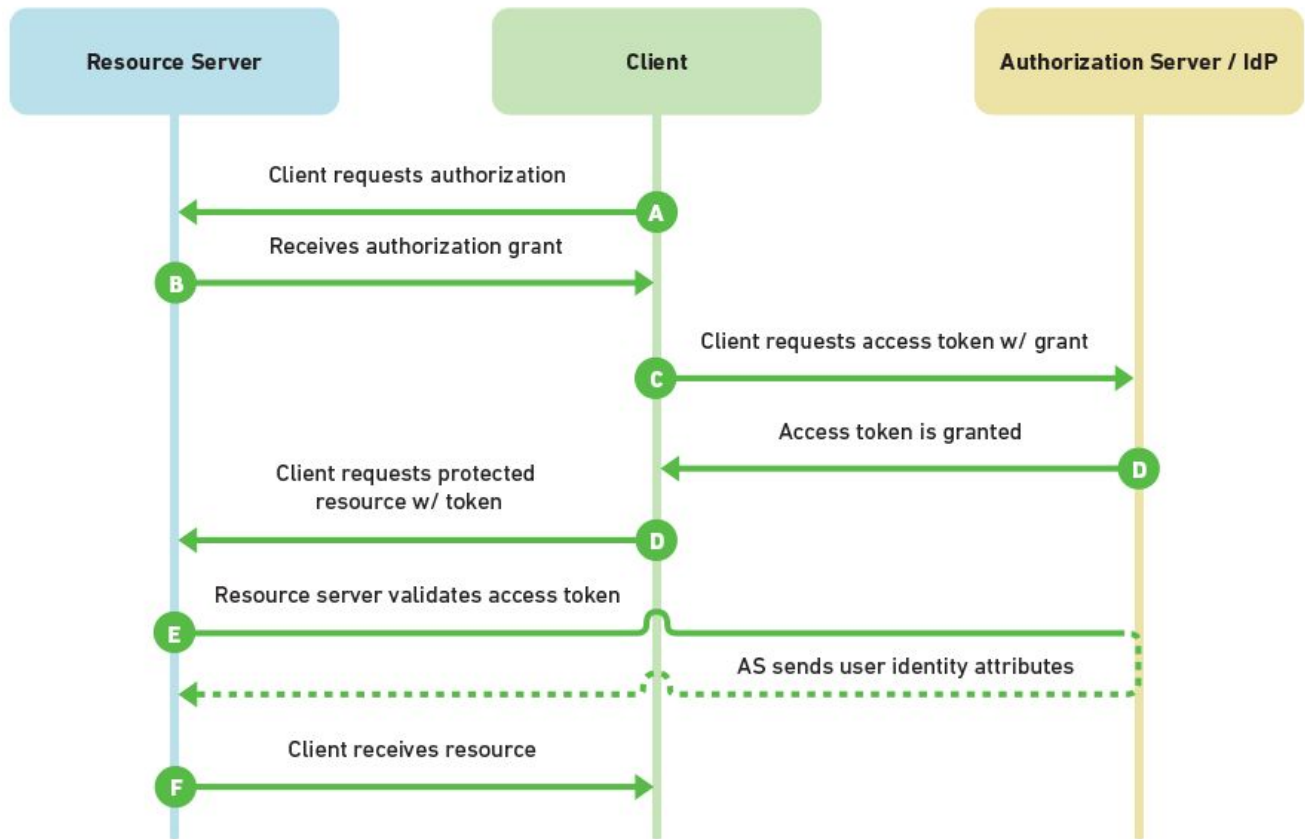


Without SSO



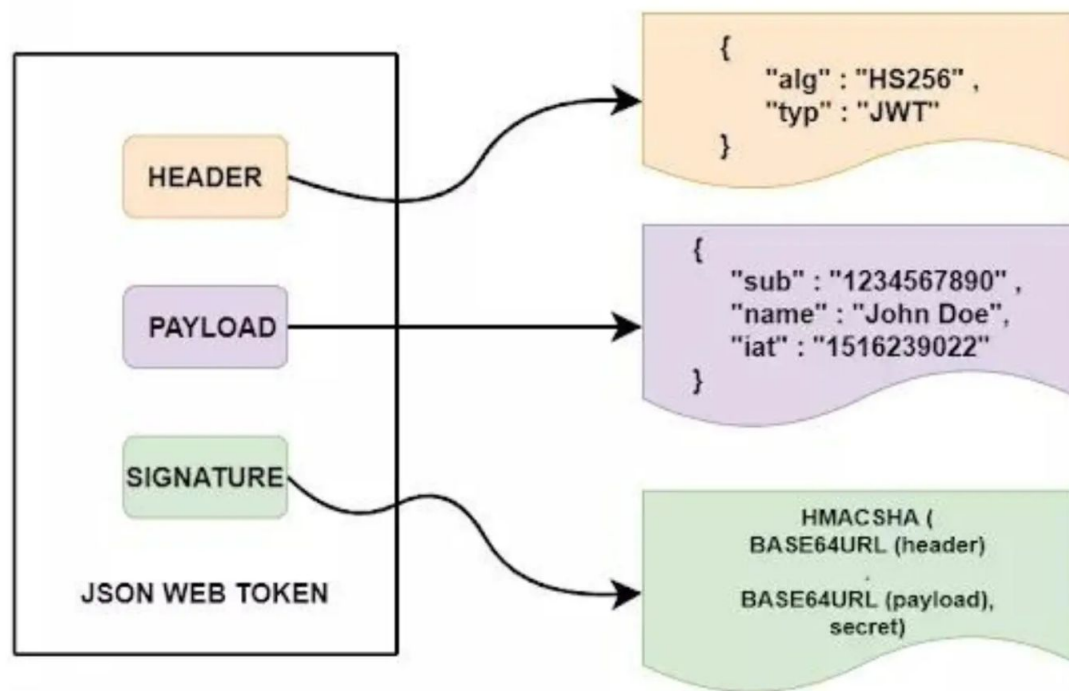
With SSO







Structure of JSON Web Token (JWT)





JWT Authnetication Workflow

